bcrypt

1. Overview

bcrypt is a **password-hashing function** designed specifically in **1999**, built around the Blowfish cipher, with a deliberately tunable **cost factor** that controls how computationally expensive each hash operation is.

Unlike general-purpose hash functions such as SHA and MD5, bcrypt was designed from the outset to be *slow*, since resisting brute-force guessing — not raw integrity verification — is the entire point of a password-hashing function.

bcrypt at a Glance

Property	Detail
Type	Dedicated password-hashing function
Based on	The Blowfish block cipher's key-setup algorithm
Designed	1999
Salting	Built into the algorithm/format by design
Tunable cost	Yes — an exponential cost/work factor

2. The Cost Factor

bcrypt's defining feature is its **cost (or "work") factor**, which controls how many internal rounds of computation each hash requires — and critically, this scaling is **exponential**.

```
cost = 10  →  2^10  =  1,024 internal rounds
cost = 12  →  2^12  =  4,096 internal rounds   (a common current default)
cost = 14  →  2^14  = 16,384 internal rounds   (higher security, slower for both attac
ker and legitimate server)
```

Why the Cost Factor Matters

This tunability is bcrypt's core design idea:

> As hardware gets faster over time, the cost factor can simply be increased for newly stored passwords, keeping the **time required to crack** roughly

> constant even as attacker hardware improves.

A fixed-speed hash function such as SHA-256 cannot provide this same adjustable work factor.

---

3. Output Format

A bcrypt hash string **self-describes its own parameters**, which is a genuinely useful property — a stored hash carries everything needed to verify it, with no separate metadata required.

Example bcrypt Hash

```
$2b$12$R9h/cIPz0gi.URNNX3kh2OPST9/PgBkqquzi.Ss7KIUgO2t0jWMUW
|   |  └──────────────────── salt (22 chars) + hash (31 chars) ─┘
|   └── cost factor (12)
└── algorithm identifier/version ($2a$, $2b$, $2y$ — variants
     reflecting historical implementation fixes)
```

Components

Component	Meaning
`$2a$`, `$2b$`, `$2y$`	Algorithm identifier/version; the variants reflect historical implementation fixes.
`12`	Cost factor.
Salt	22-character salt embedded in the bcrypt string.
Hash	31-character bcrypt hash component.

---

4. Built-In Salting

Every bcrypt hash embeds a **unique, random salt** generated at hash time.

This is a structural guarantee of the algorithm itself, not something a developer can accidentally omit or implement incorrectly when using bcrypt correctly.

This means:

- Two users with an identical password produce completely different bcrypt hashes.
- Precomputed **rainbow-table attacks** are rendered useless.

- Every hash in a dumped set must be attacked individually, rather than allowing an attacker to crack one hash and instantly know every matching duplicate.

> **Key point:** bcrypt's built-in salting prevents identical passwords from producing identical stored hashes.

---

5. Why bcrypt Resists Brute-Force Cracking

Compared to a fast general-purpose hash such as **SHA-256 or MD5**, which can be capable of tens of billions of guesses per second on modern GPU hardware, bcrypt at a reasonable cost factor (for example, **12**) typically allows only thousands to low tens of thousands of guesses per second on the same hardware.

This can represent an enormous performance difference — often around a **six-order-of-magnitude difference**.

The result is a dramatic increase in the real-world cost and time required for an attacker to brute-force a dumped password database, even against relatively weak individual passwords.

Fast Hash vs. bcrypt

Hash Type	Primary Design Goal
MD5 / SHA-family	Fast general-purpose hashing
bcrypt	Deliberately slow password hashing
Result	bcrypt significantly increases the cost of password guessing

---

6. A Known Limitation — No Deliberate Memory-Hardness

bcrypt is **compute-hard but relatively memory-light**.

This means an attacker with access to specialized parallel hardware — including GPUs and particularly ASICs — can still run many bcrypt-cracking instances in parallel more efficiently than would be possible against a function deliberately designed to also require significant memory per guess.

This is the specific gap that motivated the later development of:

- **scrypt**
- **Argon2**

Both add deliberate **memory-hardness** on top of compute-hardness, making large-scale parallel/hardware-accelerated cracking meaningfully harder.

> **Key limitation:** bcrypt is deliberately slow, but it was not designed to impose the strong memory requirements found in newer password-hashing functions such as Argon2.

---

7. Practical Considerations

Choosing a Cost Factor

The cost factor should be set **as high as the application's legitimate authentication latency budget allows**.

A common guideline is to tune the cost factor so a single hash operation takes approximately **100–250 ms on production hardware**, and periodically re-evaluate it as hardware improves.

Input Length Limitation

The original bcrypt specification silently truncates input passwords beyond **72 bytes**.

Implementations should be aware of this limitation. Some libraries pre-hash longer inputs to work around it.

Long-Term Security and Support

Despite the memory-hardness gap relative to Argon2, bcrypt remains:

- **Widely used**
- **Well tested**
- **Considered secure when configured with an appropriate cost factor**
- Supported by libraries across virtually every major programming ecosystem

---

8. Typical Use Cases

bcrypt is commonly used for:

- **Web application user password storage**, including long-standing implementations in many frameworks such as Ruby on Rails' `has_secure_password` and many PHP frameworks.
- **Systems requiring a well-tested and broadly supported password-hashing function** where Argon2 support is not readily available.

---

9. Summary

bcrypt was one of the first algorithms purpose-built for password storage, introducing two crucial ideas:

1. **Built-in salting**
2. **A tunable, exponential cost factor** that can be increased as hardware improves

Its main modern limitation relative to **Argon2** is the absence of deliberate memory-hardness.

Nevertheless, bcrypt remains a solid, well-vetted, and widely used choice — and is **vastly preferable to general-purpose fast hash functions such as MD5 and SHA-family algorithms for storing passwords**.

Quick Reference

Feature	bcrypt
Purpose	Password hashing
Salt	Built in
Cost factor	Tunable and exponential
Designed to be slow	✅ Yes
Memory-hard	❌ No
72-byte limitation	⚠️ Yes
Modern alternative	Argon2
Better than MD5/SHA for passwords	✅ Yes